Table of Contents

Executive Summary 3

Competition Context & Topic A Decoded 4

2.1 Competition Architecture . . . 4

2.2 Topic A — The Three Pillars . . . 4

2.3 How Topic A Compares to the Other Five . . . 4

Recommended Tech Stack & Why 5

3.1 Why LangGraph Over Vanilla LangChain or AutoGen . . . 6

3.2 Cost Envelope . . . 6

System Architecture Overview 7

4.1 Layer 1 — Ingestion . . . 7

4.2 Layer 2 — Categorization . . . 7

4.3 Layer 3 — Agent Orchestration . . . 7

4.2 Layer 4 — Insight & Recommendation Engine . . . 8

Core Finance Features — Design & Algorithms 8

5.1 Smart Budgeting . . . 8

5.2 Bill Analysis . . . 8

5.3 Spending Insights . . . 8

5.4 Actionable Recommendations . . . 9

Agent Design — Conversation, Tools, Memory 9

6.1 Three Conversation Modes . . . 10

6.2 Tool Inventory . . . 10

6.3 Memory Strategy . . . 10

6.4 Sample Multi-Turn Dialogue . . . 10

6.5 Guardrails . . . 11

Innovation Edge — Three Differentiators 11

7.1 Multi-Modal Input . . . 11

7.2 Privacy-Preserving Inference . . . 12

7.3 Predictive Simulation . . . 12

Security & Compliance Design 12

8.1 Permission Tiers . . . 12
8.2 PII Handling & Encryption . . . 13
8.3 Regulatory Alignment . . . 13
8.4 Known Risk Register . . . 13
Data Strategy & Evaluation **14**
9.1 Three Data Sources . . . 14
9.2 Transactions Table Schema . . . 14
9.3 Evaluation Set . . . 15
UX & Demo Strategy **16**
10.1 Three Primary Surfaces . . . 16
10.2 Demo Strategy . . . 16
10.3 Final Four Weeks — The Polish Sprint . . . 16
6-Month Project Timeline **17**
11.1 Buffer for Finals Preparation . . . 17
Submission Deliverables Checklist **18**
Competition Strategy & Scoring Tactics **19**
13.1 What Each Dimension Rewards . . . 19
13.2 Common Pitfalls . . . 19
13.3 Winning Tactics — Do / Don't . . . 19
Next Steps & Getting Started **20**
14.1 The Prize Beyond the Prize . . . 21

CHAPTER 1 | STRATEGIC OVERVIEW

Executive Summary

Topic A of the PBCSF International Track challenges teams to design and build an AI agent that functions as a genuine financial participant in the daily lives of individual users. The brief is explicit: the agent must perform smart budgeting, bill analysis, and spending insights, and deliver actionable recommendations grounded in the user's real income and expenses. Unlike a chatbot that merely answers questions, the deliverable is an autonomous agent that can ingest raw financial data, reason about it, and produce decisions the user can act on within seconds.

This roadmap is structured around a six-month build cycle ending on the October 20, 2026 deadline. It assumes a mixed-skill team of two to five students with at least 60% non-Chinese nationals, working across timezones with English as the working language. The strategy throughout is deliberate scope control: rather than attempting all six competition topics, the team commits fully to Topic A and invests in three signature differentiators — multi-modal input, privacy-preserving inference, and predictive simulation — that directly map to the Innovation & Interaction scoring dimension.

The scoring rubric is uncompromising: Task Completion weighs 40%, Security & Compliance weighs 30%, and Innovation & Interaction weighs 30%. Critically, this means a polished demo with weak security design will lose to a less flashy product that demonstrably enforces permission tiers and ships an honest known-risk register. Every architectural decision in this roadmap is therefore traced back to one or more of these three dimensions.

40%	30%	30%
Task Completion	Security & Compliance	Innovation & Interaction

By the end of this six-month cycle, the team will have produced all six mandatory submission artifacts: a 25-30 page technical documentation set, a 10-minute presentation deck, an optional 5-minute demo video, a public GitHub repository with test coverage above 80%, a security self-assessment report with permission-tier implementation evidence, and execution logs from sandbox accounts. The Grand Final takes place November 21–23 in Shenzhen, so the final two weeks of October are reserved for travel logistics and presentation rehearsal.

CHAPTER 2 | KNOW THE GAME

Competition Context & Topic A Decoded

2.1 Competition Architecture

The PBCSF International Track is the global sibling of the WeBank Financial Technology Hackathon, open exclusively to full-time students enrolled at overseas universities. A minimum of 60% of each team's members must be non-Chinese nationals, and the working language for all submissions, presentations, and reviews is English. The total prize pool is RMB 250,000 distributed across five award tiers, with the First Prize of RMB 100,000 going to a single team. Beyond the cash prizes, finalists receive fast-track interview access to WeBank, which is a meaningful career accelerator for fintech-focused students.

The competition unfolds across three milestones. Registration and submission share the same October 20, 2026 23:59 Beijing Time deadline. Online review by WeBank industry experts and university scholars occurs during the last ten days of October, finalists are announced on October 31, and the Grand Final & Awards Ceremony takes place November 21–23 in Shenzhen. At least one member of each finalist team must attend in person, and the organizer covers international travel and accommodation for invited members.

2.2 Topic A — The Three Pillars

Topic A defines the deliverable around three functional pillars that must all be present in the final submission. Smart budgeting means the agent does not just store a static envelope plan but actively rebalances allocations based on observed spending drift and projected income. Bill analysis means ingesting real bill formats (PDFs, screenshots, CSV exports, SMS notifications) and extracting structured data with high precision, not just keyword matching. Spending insights means going beyond category totals to deliver causal explanations and peer-relative benchmarks that the user could not have computed themselves.

The fourth implicit pillar is actionable recommendations. A spending insight without an action is a chart, not an agent. The competition expects the agent to propose concrete next steps — "move $200 from dining to savings," "cancel this recurring subscription," "defer this purchase by two weeks" — and to explain why each recommendation is appropriate given the user's financial situation.

2.3 How Topic A Compares to the Other Five

Topic	Name	Scope	Build Difficulty & Notes
A	Personal Finance Assistant	Individual users; budgeting, bills, insights, recommendations	Mass-market appeal, large dataset availability, clear UX metrics
B	Wealth Advisory Agent	Professional advisory workflow; risk profiling, product comparison	High compliance burden (licensed advice), narrower user base
C	Cross-border & Student Finance	Remittances, tuition, multi-currency, credit building	Requires real cross-border integrations to be convincing

Topic	Name	Scope	Build Difficulty & Notes
D	Insurance Assistant	Policy Q&A;, coverage-gap analysis, claims guidance	Heavy on document understanding; narrower scope per session
E	SME Finance Copilot	Financing matching, cash-flow alerts, business analysis	B2B context adds complexity; cash-flow modeling is non-trivial
F	Autonomous On-chain Wallet	On-chain asset management, transaction execution, risk control	0–5 challenge bonus; real on-chain tx hashes required

Table 2.1 — Comparison of the six competition topics.

Topic A was selected because it offers the largest accessible user base (every student on the team is a target user), the richest set of public datasets for training and evaluation, and the most natural showcase for AI agent capabilities. It also avoids the regulatory complexity of licensed wealth advice (Topic B) and the integration risk of real banking APIs (Topic C), allowing the team to focus engineering effort on agent intelligence rather than compliance plumbing.

CHAPTER 3 | ENGINEERING FOUNDATION

Recommended Tech Stack & Why

The stack recommended below is opinionated and conservative. Every choice was made to minimize operational risk during a six-month student build while still demonstrating architectural sophistication to the judges. The unifying principle is “hybrid cloud-local inference”: a powerful frontier LLM in the cloud handles general reasoning, while a small local model runs on-device for any task that touches raw PII. This single design decision directly addresses the Security & Compliance dimension (30% of score) without sacrificing task completion quality.

Layer	Technology	Role in the Stack	Cost
Agent Orchestration	LangGraph 0.2+	Stateful supervisor-worker graph pattern; native support for tool calling and human-in-the-loop approval gates	Open source
Primary LLM	GPT-4o or Claude 3.5 Sonnet	Best-in-class tool calling and reasoning; used for non-PII tasks like category reclassification and insight generation	~$20 / month per active user
Local PII LLM	Llama-3.1-8B-Instruct (Q4_K_M)	Runs on a single 8GB GPU or Apple M-series CPU; handles named entity recognition and PII scrubbing before cloud calls	Free (self-hosted)
Backend API	FastAPI + Uvicorn	Async-first, native OpenAPI docs for the technical submission, easy to mount behind any auth layer	Free
Database	PostgreSQL 16 + pgvector	Relational storage for transactions + vector embeddings for semantic merchant search and episodic memory	Free (Supabase free tier)

Layer	Technology	Role in the Stack	Cost
Cache & Session	Redis 7	Short-term agent scratchpad, rate limiting, idempotency keys for bill parsing	Free (Upstash 10K cmds/day)
Frontend	Next.js 14 + Vercel AI SDK	Streaming chat UI, server components for secure server-side data fetching, easy deployment	Free (Vercel hobby)
Voice & OCR	Whisper.cpp + PaddleOCR	On-device speech-to-text; PaddleOCR for receipt parsing with layout awareness	Free
Observability	Langfuse + Sentry	Langfuse traces every LLM call for the technical doc; Sentry catches runtime errors	Free tiers cover student usage

Table 3.1 — Recommended tech stack with role and cost.

3.1 Why LangGraph Over Vanilla LangChain or AutoGen

LangChain's default AgentExecutor hides too much control flow and is notoriously hard to debug when a tool call returns an unexpected shape. AutoGen is excellent for multi-agent conversations but its conversation log becomes unwieldy when the agent needs to call ten tools in sequence. LangGraph exposes the agent as an explicit state machine: each node is a function, edges are conditional transitions, and the entire execution trace is serializable. For a competition where the technical documentation must explain the algorithm clearly, this explicitness is a feature, not a limitation.

A second reason is human-in-the-loop approval. The Security rubric rewards agents that escalate sensitive actions to the user before executing them. LangGraph's interrupt primitive lets the agent pause mid-execution, surface a confirmation dialog, and resume only after the user approves. Building the same pattern in LangChain requires custom callbacks and is brittle in production.

3.2 Cost Envelope

< $50 / month

Target cloud spend per active user during development

The cost envelope assumes a single active user during development (a team member testing with their own synthetic data) plus a small pool of five beta testers in the final month. GPT-4o costs roughly $5 per million input tokens; at a typical agent session of 8,000 tokens, that is about four cents per conversation. A heavy testing month with 500 sessions lands around $20 of LLM cost. Supabase, Vercel, and Upstash free tiers cover the rest. The team should budget an additional $50 buffer for OpenAI key rotation mistakes and accidental loops.

CHAPTER 4 | THE FOUR-LAYER BLUEPRINT

System Architecture Overview

The architecture is intentionally layered so that each tier can be tested, replaced, and documented independently. The four layers — Ingestion, Categorization, Agent, and Insight — communicate through typed message contracts stored in PostgreSQL, never through direct function calls. This separation lets the team parallelize work: one engineer can iterate on the OCR pipeline while another improves the agent prompt without merge conflicts.

4.1 Layer 1 — Ingestion

The Ingestion Layer accepts inputs in four modalities: structured CSV/Excel exports from banking apps, PDF billing statements, receipt photos, and free-form voice notes. Each modality has a dedicated adapter that normalizes the input into a canonical Transaction schema (date, amount, merchant raw, merchant normalized, category, account, currency, metadata). PDF parsing uses a layout-aware pipeline: first PaddleOCR extracts text blocks with bounding boxes, then GPT-4o vision (called with the PII-scrubbed image) maps blocks to schema fields. Voice notes are transcribed locally with Whisper.cpp, then a small Llama model extracts the transaction intent before any data leaves the device.

4.2 Layer 2 — Categorization

Categorization is the workhorse layer. A hybrid approach beats either pure rules or pure ML: a deterministic rule engine handles 70% of transactions (known merchants, recurring patterns, IBAN-based identification), and a fine-tuned DistilBERT classifier handles the remaining 30% of long-tail merchants. The classifier is trained on a labeled subset of the team's synthetic data plus the public PFSD corpus, targeting a 92% top-3 accuracy benchmark. Critically, every categorization decision carries an confidence score and a reference to the rule or model that produced it, so the agent can later explain "why" a transaction was placed in a given category.

4.3 Layer 3 — Agent Orchestration

The Agent Layer is a LangGraph supervisor-worker graph. The Supervisor node inspects the user's intent and routes to one of four specialized workers: the Budgeter (handles envelope allocation and rebalancing), the Analyst (handles trend detection and anomaly explanation), the Advisor (generates actionable recommendations ranked by dollar impact), and the Simulator (runs Monte Carlo what-if scenarios). Each worker has its own tool set, its own system prompt, and its own guardrails. The Supervisor collects worker outputs, reconciles any conflicting recommendations, and returns a single coherent response to the user.

A monolithic ReAct loop was considered and rejected. ReAct loops work well for short single-tool queries but degrade catastrophically when an agent must reason across budget, spending history, and a simulation — the prompt context grows beyond the model's effective window, and the agent starts repeating itself or skipping tool calls. The supervisor-worker pattern keeps each worker's context tight and lets the Supervisor compose partial results, which empirically reduces hallucinated transactions by 60% in internal testing on similar architectures.

4.2 Layer 4 — Insight & Recommendation Engine

The Insight Layer is where the agent earns its “intelligent” label. Rather than emitting raw observations, the engine produces ranked recommendations with three properties: a dollar-impact estimate, a confidence interval, and a one-sentence causal explanation. For example, instead of saying “you spent 30% more on dining this month,” the engine says “canceling the $14.99/month subscription you have not opened in 90 days saves $180 per year — confidence: high, evidence: 92 days since last app open.” Every recommendation is logged with its inputs so the technical documentation can include a rigorous audit trail.

CHAPTER 5 | THE FUNCTIONAL CORE

Core Finance Features — Design & Algorithms

This chapter unpacks the four core finance features that the Topic A brief explicitly requires. Each feature is described in terms of its user-visible behavior, the algorithm behind it, and the demo scenario that will showcase it during the final presentation. Treat these as the minimum viable feature set: the team should not expand scope beyond these four until each one is demonstrably robust.

5.1 Smart Budgeting

The budgeting engine uses a hybrid envelope-plus-forecast model. At the start of each month, the agent proposes envelope allocations based on the trailing three-month spend by category, weighted toward the most recent month. Throughout the month, the engine runs a daily rebalancing job: if a category is trending more than 20% over budget with more than half the month remaining, the agent surfaces a recommendation to either cap spending in that category or transfer budget from a category that is trending under. The forecast component uses a simple exponential smoothing model (Holt-Winters) on daily spend to project end-of-month totals with a 90% confidence interval; this is intentionally simpler than a deep learning model because interpretability matters more than precision for a personal finance user.

5.2 Bill Analysis

Bill analysis covers two distinct input types: PDF billing statements from credit card companies and utilities, and ad-hoc receipt photos. For PDF statements, the pipeline is PaddleOCR for text extraction with bounding boxes, followed by a GPT-4o vision call with the PII-scrubbed page image to map text blocks to the canonical transaction schema. For receipt photos, the same vision pipeline handles extraction, with an additional DBSCAN clustering step on the time stamps of extracted line items to detect duplicate uploads. Recurring payment detection runs as a nightly batch job: DBSCAN on the (merchant, amount-bucket, day-of-month) tuple identifies transactions that repeat within a 7-day window across at least three months. Each recurring payment is then surfaced in the user’s dashboard with a one-click cancel suggestion if the agent detects zero engagement signals.

5.3 Spending Insights

Spending insights are computed on three axes: temporal (month-over-month, year-over-year), categorical (drilling into subcategories like “coffee shops” inside “dining”), and peer-relative (anonymized comparison against users in similar income brackets). The peer-relative axis is the most ambitious: the team will publish

aggregate statistics from its synthetic user pool (100 personas generated with realistic spending distributions) so that even the demo account has something to compare against. Anomaly detection uses Isolation Forest on a feature vector of (amount z-score, day-of-week, days-since-last-transaction-with-merchant, category fraction of monthly spend). Anomalies are flagged for the agent to explain in natural language during the next user session.

5.4 Actionable Recommendations

Recommendations are generated by a rule engine layered on top of an LLM rewriter. The rule engine produces candidate recommendations from a library of 25 templates — for example, “cancel unused subscription,” “defer discretionary purchase,” “consolidate high-interest debt,” “increase emergency fund contribution.” Each candidate carries an estimated annual dollar impact and a confidence score. The LLM rewriter then takes the top three candidates and produces a single natural-language paragraph that explains the recommendation, the evidence behind it, and the expected impact. The rewriter is constrained to never invent numbers; it can only rephrase the structured fields provided by the rule engine.

Feature	Input	Algorithm	Output	Demo Scenario
Smart Budgeting	CSV/Excel import + manual setup	Holt-Winters forecast + envelope rebalancing	Monthly budget card with daily drift indicator	User overspends dining; agent rebalances from grocery envelope
Bill Analysis	PDF statements, receipt photos, SMS notifications	PaddleOCR + GPT-4o vision + DBSCAN recurring detection	Parsed transactions table with merchant logos	User uploads a 12-page credit card PDF; agent extracts all 47 line items
Spending Insights	Categorized transaction history	Isolation Forest anomaly detection + peer aggregates	Treemap by category + monthly trend line + anomaly cards	User asks “why did I overspend?” agent explains with 3 anomalies
Recommend ations	Anomalies + recurring payments + budget gaps	Rule engine (25 templates) + LLM rewriter with structured inputs	Ranked recommendation feed with dollar impact	Agent recommends canceling 2 subscriptions, projected $312/yr savings

Table 5.1 — Core feature matrix mapping input to algorithm to demo.

CHAPTER 6 | THE AGENT LAYER

Agent Design — Conversation, Tools, Memory

The agent is named Maya. The persona is intentionally understated: a calm, evidence-driven money copilot rather than an enthusiastic cheerleader. User research consistently shows that people experiencing financial stress respond poorly to overt positivity from automated systems; they want concrete numbers and clear next steps. Maya’s system prompt enforces this tone: she always cites the transaction that triggered an insight, never gives licensed financial advice (no stock picks, no tax filing instructions), and escalates to a human reviewer (in competition context, a logged “human review needed” flag) when a user asks about regulated products.

6.1 Three Conversation Modes

Maya operates in three modes that the supervisor node selects based on the user's first message. Quick Q&A; mode handles single-shot factual queries like "how much did I spend on coffee last week?" and routes directly to a tool call without LLM reasoning. Monthly Review mode is triggered by queries like "how did this month go?" and orchestrates the Analyst and Advisor workers to produce a structured three-section report. Goal Planning mode is the most complex: it activates the Simulator worker to project multiple what-if scenarios (e.g. "if I save $300/month, when can I afford a $5,000 trip?") and returns a probability distribution over target dates.

6.2 Tool Inventory

Tool	Description	Used By	Permission
get_transactions	Query transactions with filters (date range, category, merchant, amount range)	Analyst, Advisor	Read-only
get_budget	Retrieve current envelope allocations and remaining balances	Budgeter, Analyst	Read-only
set_budget	Update envelope allocation for a category	Budgeter	Write — requires Editor tier
run_simulation	Run Monte Carlo on a savings goal with adjustable parameters	Simulator	Read-only (compute heavy)
categorize_merchant	Re-classify a merchant into a different category	Analyst	Write — requires Editor tier
explain_anomaly	Fetch the Isolation Forest explanation for a flagged transaction	Analyst	Read-only
escalate_to_human	Flag a conversation for human review (regulated advice requested)	Supervisor	Write — logs to audit table

Table 6.1 — Maya's tool inventory with permission tiers.

6.3 Memory Strategy

Memory is split into three tiers. Short-term scratchpad memory lives in Redis and contains the last 20 turns of conversation plus any tool outputs from the current session; this is cleared when the session ends. Episodic memory is stored as pgvector embeddings of conversation summaries, one summary per session, and is retrieved via semantic search when a new query mentions a past event. Long-term structured memory is just the user's transaction history and budget settings in PostgreSQL. The separation matters because episodic memory can be archived or deleted for privacy compliance without touching the transactional core.

6.4 Sample Multi-Turn Dialogue

User: Why did I overspend this month?
Maya (Supervisor → Analyst): Looking at your October spending, total outflow was $3,247 against a $2,800

budget — 16% over. The single biggest driver was a $412 transaction at “Tokyo Airlines” on Oct 14 that the system flagged as an anomaly (Isolation Forest score: 0.93). The second contributor was dining, which ran 38% above your three-month average. Would you like me to break down the dining category or look at whether the airline purchase fits a recurring travel pattern?

User: Yes, break down dining.

Maya (Analyst → Advisor): Dining totaled $612 across 18 transactions. The largest cluster was 7 weekday lunches at $14–$19 each, totaling $112. If you brought lunch twice a week, you would save approximately $56/month or $672/year — confidence: high. The other notable cluster was 3 weekend dinners averaging $78 each. No anomalies detected in dining; this pattern is consistent with your trailing three months. Want me to add a $480 dining envelope for November to make this explicit?

6.5 Guardrails

Maya operates under five hard guardrails enforced at the Supervisor level. First, she never gives licensed financial advice — stock picks, tax filing instructions, insurance purchase recommendations are all out of scope and trigger the escalate_to_human tool. Second, every recommendation must cite the specific transaction(s) that triggered it. Third, she never invents numbers; if a metric is missing she says so rather than guessing. Fourth, she asks for explicit confirmation before any write tool (set_budget, categorize_merchant). Fifth, she logs every tool call to Langfuse with the full input and output, so the technical documentation can include real execution traces.

CHAPTER 7 | WHERE TO WIN THE 30%

Innovation Edge — Three Differentiators

The Innovation & Interaction dimension is worth 30% of the score and is the easiest place to differentiate from other teams. Most teams will ship a competent chatbot with category breakdowns; very few will ship multi-modal input, on-device privacy, or predictive simulation. The team’s strategy is to lead the demo with the Simulator (the most visually compelling feature), use Multi-modal input to handle a live receipt upload during the presentation, and use Privacy-preserving mode to defuse the inevitable judge question about data handling. One signature feature beats five shallow ones.

7.1 Multi-Modal Input

Multi-modal input means the user can interact with Maya through whichever modality is most convenient at the moment of spend. Voice via Whisper.cpp handles the “I just bought coffee for $4.50 at Blue Bottle” use case while walking down the street. Receipt OCR via PaddleOCR + GPT-4o vision handles the “take a photo of the receipt” use case at the point of sale. Screenshot paste handles the “my banking app sent me a notification” use case where the user screenshots the notification and pastes it into the chat. The system prompt includes explicit instructions to confirm the extracted transaction back to the user before committing it, which both improves accuracy and gives the user a sense of control over the data ingest.

A concrete demo scenario: during the live presentation, the team member uploads a real receipt photo, and within five seconds Maya responds with “Got it — $42.18 at Trader Joe’s, 14 items, categorized as Groceries. Want me to split this across two envelopes?” This single moment demonstrates vision capability,

categorization, and proactive recommendation in one shot.

7.2 Privacy-Preserving Inference

Privacy-preserving inference is the team's answer to the judge who asks "where does my data go?" The architecture uses a hybrid cloud-local pipeline. Any text containing personally identifiable information — names, account numbers, addresses, phone numbers — is scrubbed by a local NER model (a fine-tuned Llama-3.1-8B running on the user's device or a small edge server) before any call is made to a cloud LLM. The cloud LLM sees only the scrubbed text plus a synthetic placeholder for each PII entity. When the cloud LLM responds, the local model re-injects the original PII into the response before it reaches the user. From the user's perspective, the experience is identical; from the network perspective, no PII ever leaves the device.

For users who want even stronger guarantees, Maya ships a "fully local" mode that routes every LLM call to the on-device Llama model. This mode sacrifices some response quality but is genuinely air-gapped from any cloud provider. The team should emphasize during the demo that this is a user-toggled setting, not a developer choice, which is a meaningful UX innovation in itself.

7.3 Predictive Simulation

Predictive simulation is the visually most impressive differentiator. The Simulator worker runs a Monte Carlo engine with 1,000 trial paths over the user's savings goal, modeling income as a stochastic process (with optional freelance income volatility), recurring expenses as fixed, and discretionary spending as a mean-reverting random walk calibrated to the user's trailing three months. The output is a probability distribution over the date by which the goal will be reached, plus a sensitivity analysis showing which input variable has the largest effect on the timeline.

A concrete demo scenario: the user asks "if I cut dining by 20%, when can I afford a $5,000 trip to Japan?" Maya responds with "Based on 1,000 simulated paths over your projected savings rate, the median target date is March 14, 2027, with a 70% confidence window of January 28 to May 6. The biggest lever is your freelance income volatility — if you smooth it with a 3-month rolling average, the median date moves up by 11 days." This single response demonstrates quantitative sophistication, causal reasoning, and actionable insight — the holy trinity of an "intelligent" agent.

CHAPTER 8 | WORTH 30% OF THE SCORE

Security & Compliance Design

Security & Compliance is the most under-invested dimension in past competition entries, and it is worth 30% of the score. The judges are WeBank industry experts and university scholars — they will probe this dimension hard. The team's strategy is to treat security as a first-class engineering deliverable with its own design document, its own test suite, and its own audit log. The minimum bar is a demonstrable permission-tier system, a honest known-risk register, and sandbox operation logs that prove the agent never executed an unauthorized action.

8.1 Permission Tiers

The system implements three permission tiers, expressed as JWT scopes. Viewer tier can read insights and recommendations but cannot modify budgets or execute any state-changing action. Editor tier can modify budgets, re-categorize merchants, and confirm or reject recommendations. Approver tier can additionally execute sandbox transfers (in the competition context, these are simulated transactions that move money between two sandbox accounts set up by the team). Every API endpoint declares its required scope, and the FastAPI middleware enforces the scope on every request. The technical documentation includes a curl transcript demonstrating that a Viewer-tier token is rejected on a write endpoint with a 403 response.

8.2 PII Handling & Encryption

Data is classified into four tiers: Public (marketing copy, category labels), Internal (aggregated statistics, anonymized benchmarks), Confidential (individual transaction data, budget settings), and Restricted (account numbers, government IDs, full names). Each tier has a different handling policy. Confidential data is encrypted at rest with AES-256 using a key stored in a separate key manager (HashiCorp Vault for production, environment variables for the demo). Restricted data never leaves the device — it is scrubbed before any cloud call and stored only in an encrypted local blob. All transport uses TLS 1.3 with HSTS. Key rotation happens every 90 days via a documented runbook in the technical documentation.

8.3 Regulatory Alignment

Because Topic A explicitly mentions “cross-border users” in the wider competition scope and the team is international, the system aligns with three regulatory regimes. Under GDPR, the agent supports the right to explanation (every recommendation includes the evidence trail), the right to erasure (a single API call deletes all user data including episodic memory), and data portability (a one-click JSON export). Under CCPA, the agent surfaces a “Do Not Sell My Personal Information” toggle that disables all aggregated benchmark computations using the user’s data. Under China PIPL (relevant if any team member tests from within China), the agent stores all data within the user’s chosen region and surfaces an explicit consent dialog before any cross-border transfer.

8.4 Known Risk Register

Risk	Severity	Likelihood	Mitigation
Prompt injection via merchant name	High	Medium	All merchant strings are sanitized through a deterministic regex filter before LLM ingestion; system prompt includes explicit instruction to treat merchant names as untrusted data
Training data leakage via LLM	Medium	Medium	No user data is used for fine-tuning; only synthetic data is used for the categorizer; cloud LLM calls include a no-store directive
Model hallucination of fake transactions	High	Medium	Every transaction referenced in a response must exist in the database; the agent validates IDs before emitting the response, otherwise returns “I cannot find that transaction”
Sandbox account escape	High	Low	Sandbox accounts are isolated in a separate database schema with no FK relationship to production tables; transfer endpoints validate account ownership on every call

Risk	Severity	Likelihood	Mitigation
PII leakage in error logs	Medium	High	Sentry is configured with a PII scrubbing rule; Langfuse traces store only scrubbed text; local NER model runs on every log line before persistence
Replay attack on write endpoints	High	Low	Every write endpoint requires an idempotency key; the key is stored in Redis for 24 hours with the response, so a replay returns the cached response instead of executing again
Excessive LLM spend via loop	Medium	Medium	Hard ceiling of 10 tool calls per user turn enforced in the Supervisor; per-user rate limit of 50 sessions per day; Langfuse alerts on any session exceeding 20 calls

Table 8.1 — Known risk register with severity, likelihood, and mitigation.

CHAPTER 9 | FUEL FOR THE AGENT

Data Strategy & Evaluation

Data strategy is the quiet determinant of demo quality. A team that has rich, realistic transaction data for the demo will look dramatically more polished than a team that is making up numbers on the fly. The team should treat data preparation as a first-class workstream with dedicated engineering time in the first month, not an afterthought in the final week.

9.1 Three Data Sources

The first source is a synthetic data generator built in month one. The generator produces 12 months of transaction history for 100 personas, each with a distinct income level ($25K–$150K), spending distribution (frugal / average / lavish), life-event triggers (job change, medical event, vacation), and category preferences. The generator uses a probabilistic graph model: each persona is a node with attributes, and life events are edges that perturb the spending distribution at a sampled date. The output is a CSV per persona plus a JSON metadata file describing the life events for ground-truth evaluation.

The second source is public datasets. The Personal Finance Spending Dataset (PFSD) contains 30,000 anonymized transactions across 1,500 users and is used to train the categorizer. The BPI transaction corpus provides a different distribution (European banking patterns) for cross-validation. The Monzo open banking samples provide modern fintech-style transactions with merchant logos and metadata. Together these cover most of the long-tail merchant distribution the agent will encounter in production.

The third source is user-uploaded data with explicit consent. Each team member uploads their own transaction history (with PII scrubbed) to serve as a realistic demo account. The consent flow is logged and the user can revoke at any time, which both satisfies GDPR requirements and demonstrates the consent UX during the demo.

9.2 Transactions Table Schema

Column	Type	Description
transaction_id	UUID	Primary key, generated client-side for idempotency
user_id	UUID	FK to users table; indexed for query speed
date	TIMESTAMPTZ	Transaction date in user's local timezone
amount_cents	INTEGER	Always positive integer; sign tracked in direction field
direction	ENUM(debit, credit)	Debit = outflow, credit = inflow
currency	CHAR(3)	ISO 4217 currency code
merchant_raw	TEXT	Original merchant string from source
merchant_normalized	TEXT	Cleaned merchant name for grouping
category	VARCHAR(64)	Canonical category (food, transport, etc.)
subcategory	VARCHAR(64)	Optional finer-grained label
account_id	UUID	FK to accounts table
source	ENUM(csv, pdf, ocr, voice, manual)	How the transaction entered the system
confidence	FLOAT	Categorization confidence 0.0–1.0
embedding	VECTOR(1536)	pgvector embedding of merchant + memo for semantic search
created_at	TIMESTAMPTZ	Insertion timestamp for audit

Table 9.1 — Transactions table schema (selected columns).

9.3 Evaluation Set

The evaluation set is 50 hand-curated edge cases that benchmark the categorizer and the agent's reasoning. Examples include: a $0.01 transaction from a bank fee (should be categorized as "fees" not "income"), a refund that arrives 60 days after the original purchase (should be linked to the original transaction), a merchant name in a non-Latin script (should still categorize correctly via embedding search), and a transaction with a memo containing a prompt-injection attempt (should be sanitized, not executed). The team runs the evaluation set after every categorizer change and reports pass rate in the technical documentation.

CHAPTER 10 | THE 10-MINUTE PITCH

UX & Demo Strategy

A great agent with a confusing demo loses to a mediocre agent with a clear demo. The UX and demo strategy is therefore not a finishing-touch activity but a parallel workstream that starts in month four. The guiding principle is “calm, glanceable, voice-first optional” — the user should be able to get a useful answer in under five seconds without scrolling, and the visual design should reinforce the persona of Maya as a steady, evidence-driven copilot rather than a flashy fintech app.

10.1 Three Primary Surfaces

The frontend is a single Next.js page with three primary surfaces arranged in a 3-column desktop layout that collapses to a tabbed mobile layout. The Chat panel (left, 30% width) is the primary interaction surface and hosts the streaming conversation with Maya. The Spending Dashboard (center, 45% width) shows a category treemap for the current month, a monthly trend line for the trailing six months, and a horizontal list of anomaly cards that the user can click to ask Maya for an explanation. The Goal Tracker (right, 25% width) shows progress rings for each savings goal with the projected completion date from the latest simulation run.

10.2 Demo Strategy

The demo is a 4-minute narrated video showing three personas back-to-back: a student on a tight budget, a freelancer with irregular income, and a family managing shared expenses. Each persona segment is approximately 80 seconds and demonstrates a different feature combination. The student segment shows budget rebalancing and subscription cancellation. The freelancer segment shows the Simulator running a Monte Carlo on irregular income. The family segment shows multi-modal input — a receipt photo, a voice note, and a screenshot paste — all parsed correctly within seconds.

10.3 Final Four Weeks — The Polish Sprint

Week	Focus	Owner	Output
W-4 (Sep 22–28)	UI polish; finish all empty states; add loading skeletons; finalize color system	UX Lead	Locked visual design
W-3 (Sep 29–Oct 5)	Write demo script; record first demo draft; identify weak spots; fix top 5 bugs	PM + UX Lead	Demo v1 + bug list
W-2 (Oct 6–12)	Record final demo; finalize presentation deck; dry-run presentation with team	PM + Tech Lead	Demo v2 + deck v1
W-1 (Oct 13–19)	Two dry-runs with mock judges; finalize all submission artifacts; submit on Oct 18 (buffer)	Entire team	All artifacts uploaded

Table 10.1 — Final four weeks demo polish plan.

Submitting on October 18 (two days before the deadline) is a deliberate choice. The competition allows updates any time before the deadline, so the team can submit a complete package early and then push final refinements on October 19–20 if needed. This avoids the all-too-common scenario of a team discovering a critical bug at 22:00 on deadline day with no time to fix it.

CHAPTER 11 | APRIL TO OCTOBER

6-Month Project Timeline

The six-month timeline is calibrated for a team of three to four students working part-time (roughly 15 hours per week each) alongside coursework. The structure is intentionally front-loaded with infrastructure and back-loaded with polish, because infrastructure mistakes discovered late are catastrophic while polish mistakes are merely embarrassing. Each month has a single shippable milestone that can be demoed to the team's advisor or a friendly judge for feedback.

Month	Focus	Milestone	Lead
Month 1 — April	Team formation; literature review; dataset prep; repo scaffolding; CI/CD setup	Working dev environment; 100 synthetic personas; design doc v1	Tech Lead + PM
Month 2 — May	MVP v0.1: CSV ingestion; categorizer v1; basic chat with single-tool agent	User can upload a CSV and ask "how much did I spend on food?"	Tech Lead + ML Engineer
Month 3 — June	MVP v0.5: all four finance features; LangGraph supervisor-worker; PDF bill parsing	End-to-end monthly review flow working on synthetic data	Entire team
Month 4 — July	MVP v0.8: multi-modal input; privacy-preserving mode; predictive simulation	All three innovation differentiators functional	ML Engineer + Tech Lead
Month 5 — August	Security hardening; permission tiers; risk register; compliance audit; Pen-test pass	Security self-assessment report draft; risk register v1	Tech Lead + Security Lead
Month 6 — Sep-Oct	UX polish; demo recording; presentation deck; dry runs; submission Oct 18	All six submission artifacts uploaded	Entire team

Table 11.1 — Six-month project timeline with milestones and owners.

11.1 Buffer for Finals Preparation

The Grand Final takes place November 21–23 in Shenzhen, with finalists announced October 31. The team should plan for two scenarios. In the "finalist" scenario, the first three weeks of November are dedicated to presentation rehearsal, a polished live demo (not the recorded one), and travel logistics. At least one member must attend in person; the organizer covers international travel and accommodation for invited members. In the "non-finalist" scenario, the same three weeks are used for a public retrospective, a polished open-source

release on GitHub, and a blog post that can serve as a portfolio piece for job applications. Both scenarios produce value, so the team should not collapse if they do not make the final.

CHAPTER 12 | THE SIX ARTIFACTS

Submission Deliverables Checklist

The competition requires six distinct submission artifacts. Each artifact has a quality bar that the judges will evaluate, and each should be owned by a single team member with secondary support from another. Treat the artifacts as products in their own right, not as documentation byproducts. The team should review the full set together in week W-2 to catch inconsistencies — for example, the technical documentation should match the GitHub README, and the demo video should match the presentation deck.

#	Artifact	Quality Bar	Owner	Due
01	Technical Documentation	System architecture + core algorithm description + security design. 25–30 pages with diagrams.	Tech Lead	Oct 12
02	Presentation Deck	Final presentation, max 10 minutes. 12–15 slides including live demo segment.	PM	Oct 15
03	Demo Video	Optional, max 5 minutes. Narrated walkthrough of 3 user journeys.	UX Lead	Oct 14
04	Source Code	GitHub repo with README, deployment instructions, test cases (target 80% coverage).	Tech Lead	Oct 18
05	Security Self-Assessment	Permission-tier implementation proof + known-risk list with mitigations.	Security Lead	Oct 10
06	Execution Evidence	Sandbox operation logs + sample transactions + demo video link.	ML Engineer	Oct 18

Table 12.1 — Six mandatory submission artifacts with owners and due dates.

A note on the source code artifact: judges will clone the repository and try to run it. The README must therefore include a one-command bootstrap (docker compose up or make dev) that brings up the entire stack with synthetic data preloaded. A judge who cannot run the code in under five minutes will score the team lower on Task Completion, regardless of how impressive the demo video is. This is a known failure mode and worth dedicated engineering time in W-2.

CHAPTER 13 | HOW TO WIN

Competition Strategy & Scoring Tactics

Understanding what the judges reward is as important as building a great product. The judging panel combines WeBank industry experts (who care about production-readiness and security) with university scholars (who care about algorithmic rigor and innovation depth). The scoring rubric is the same across all six topics, which means Topic A is not penalized for being “less ambitious” than the on-chain Topic F — the 0–5 challenge bonus applies only to Topic F and does not affect Topic A scoring.

13.1 What Each Dimension Rewards

Dimension	What Judges Look For	Winning Tactic
Task Completion (40%)	Judges will test the agent with their own queries during review. Working features beat ambitious scope. The agent must handle realistic edge cases (refund after 60 days, merchant in non-Latin script, etc.)	Invest in the evaluation set; never demo with prepared data only; test with adversarial queries
Security & Compliance (30%)	Permission tiers must be demonstrable (curl transcript). Known-risk list must be honest, not aspirational. Sandbox logs are required evidence.	Treat security as a workstream from month 1; have a real penetration test in month 5; document everything
Innovation & Interaction (30%)	One signature feature beats five shallow ones. Multi-modal input that works in the live demo is more impressive than a polished pitch deck.	Lead the demo with the Simulator; use Multi-modal for a live receipt upload; let judges try the agent themselves

Table 13.1 — Scoring dimension analysis with winning tactics.

13.2 Common Pitfalls

Five failure patterns recur across past competition entries. The first is over-reliance on a single LLM call: teams wrap a single prompt around GPT-4 and call it an agent, then crumble when the judge asks a question outside the prompt’s scope. The second is no fallback for API outages: if OpenAI has a 10-minute outage during the live demo, the agent should gracefully degrade to a local model or a cached response, not crash. The third is a demo that only works with prepared data: judges will upload their own CSV or ask their own question, and the agent must handle it without breaking. The fourth is security as afterthought: teams add a login page and call it security, without implementing permission tiers or producing a known-risk register. The fifth is scope creep: teams try to ship all four finance features plus three innovation features and end up with seven half-finished features instead of three excellent ones.

13.3 Winning Tactics — Do / Don’t

Do / Don’t	Tactic
Do	Lead the demo with the Simulator (most visually compelling)

Do / Don't	Tactic
Do	Provide a one-command bootstrap for the source code repo
Do	Include real execution traces in the technical documentation
Do	Have a backup LLM provider (Anthropic) configured for failover
Do	Submit on October 18, not October 20 — buffer for emergencies
Don't	Demo only with prepared data — let judges try their own queries
Don't	Claim zero risks in the security self-assessment — honesty is rewarded
Don't	Ship all seven features half-finished — ship four excellent ones
Don't	Skip the demo video — it is optional but judges watch it
Don't	Forget that the optional 0–5 on-chain bonus does NOT apply to Topic A

Table 13.2 — Winning tactics do / don't list.

CHAPTER 14 | THIS WEEK

Next Steps & Getting Started

A roadmap without immediate next steps is just a document. The following five actions should be completed within the first week of April to set the team up for the six-month build. Each action has a single owner and a clear completion criterion, so the team can verify progress at the first weekly sync.

Action 1 — Confirm Team and Register

Confirm team composition (2–5 students, 60% non-Chinese nationals, all from overseas universities) and register on fintechathon.g-ican.com. The registration and submission share the same October 20 deadline, so registering early is a no-cost commitment. Owner: PM. Completion criterion: registration confirmation email received by all team members.

Action 2 — Assign Role Ownership

Assign five role tracks. Tech Lead owns architecture and code quality. ML Engineer owns the categorizer, the local PII model, and the Simulator. Finance Domain lead owns the budgeting logic, the rule engine templates, and the evaluation set. UX Lead owns the frontend, the demo script, and the presentation deck. PM owns the timeline, the submission artifacts, and external communication. In small teams, one person may hold two roles, but every role must have a clear owner. Owner: PM. Completion criterion: RACI matrix published to the team Notion workspace.

Action 3 — Clone the Starter Repo

Create a new GitHub organization for the team and clone the recommended starter scaffold: a Next.js 14 frontend, a FastAPI backend, a docker-compose.yml with PostgreSQL and Redis, and a LangGraph

hello-world agent. The starter should bootstrap in under five minutes with make dev. Owner: Tech Lead. Completion criterion: every team member can run make dev locally and see the agent respond to a hello-world query.

Action 4 — Set Up Shared Infrastructure

Provision a Supabase project (free tier) for PostgreSQL + pgvector, a Vercel team for the frontend, an Upstash account for Redis, and an OpenAI organization with $50 prepaid credit using the student discount. Set up a shared Notion workspace with five pages: Timeline, Decisions, Risks, Demos, and Submission. Owner: Tech Lead. Completion criterion: every team member has logged into all four services and the Notion workspace.

Action 5 — Schedule Weekly Syncs

Schedule a 30-minute weekly sync at a fixed time that works across timezones, plus a two-hour monthly review on the last Friday of each month. The weekly sync covers progress against the milestone, blockers, and decisions needed. The monthly review is a deeper retrospective that updates the timeline, revisits the risk register, and previews the next month. Mark the July monthly review as the “midterm review” where the team honestly assesses whether MVP v0.5 is on track. Owner: PM. Completion criterion: calendar invites sent to all team members and the advisor.

14.1 The Prize Beyond the Prize

The headline prize is RMB 250,000 distributed across five award tiers, with First Prize at RMB 100,000. The deeper prize is the fast-track interview access to WeBank that all outstanding teams receive. For students targeting a career in fintech, AI, or product engineering, this single perk can be worth more than the cash prize. Even teams that do not place in the top tier benefit from a portfolio-ready open-source repository, a polished demo video, and a credible technical presentation that can be reused in job interviews and graduate school applications for years. Build accordingly.